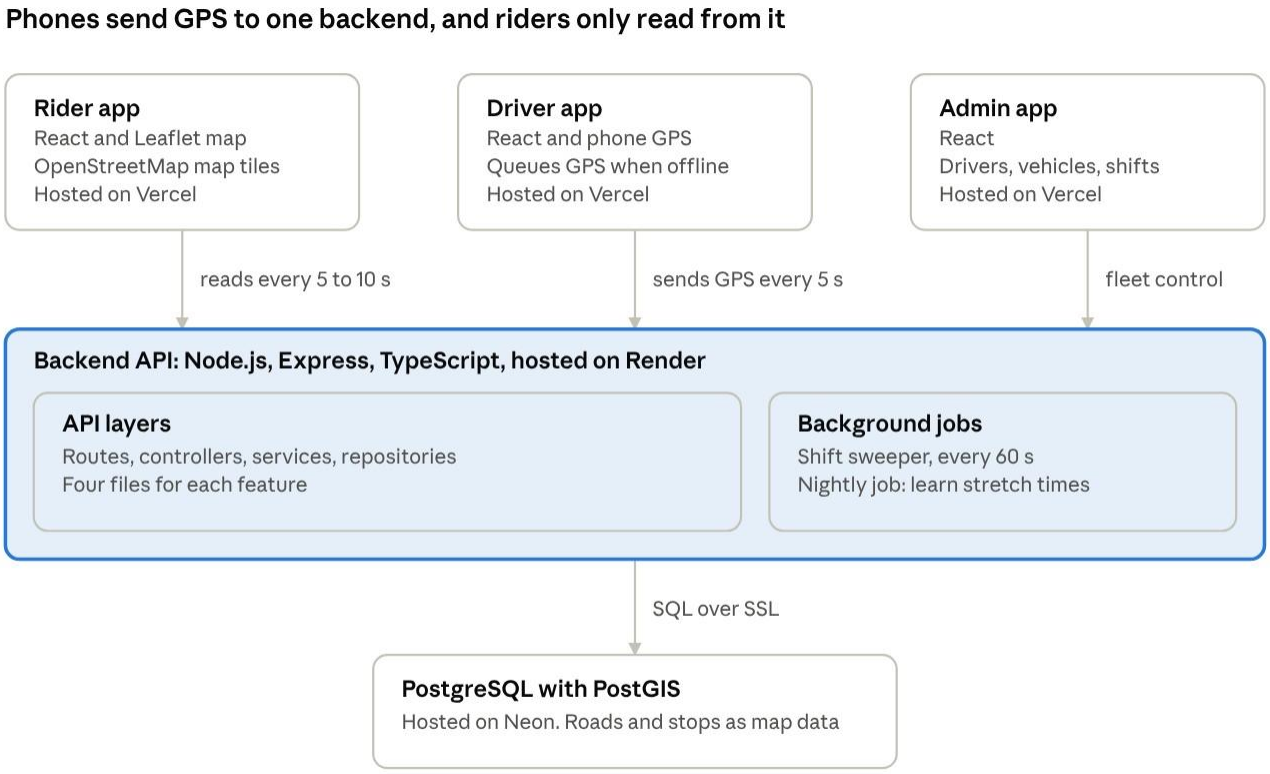
System Design - Addis Ababa Bus Tracking System

1. Architecture overview

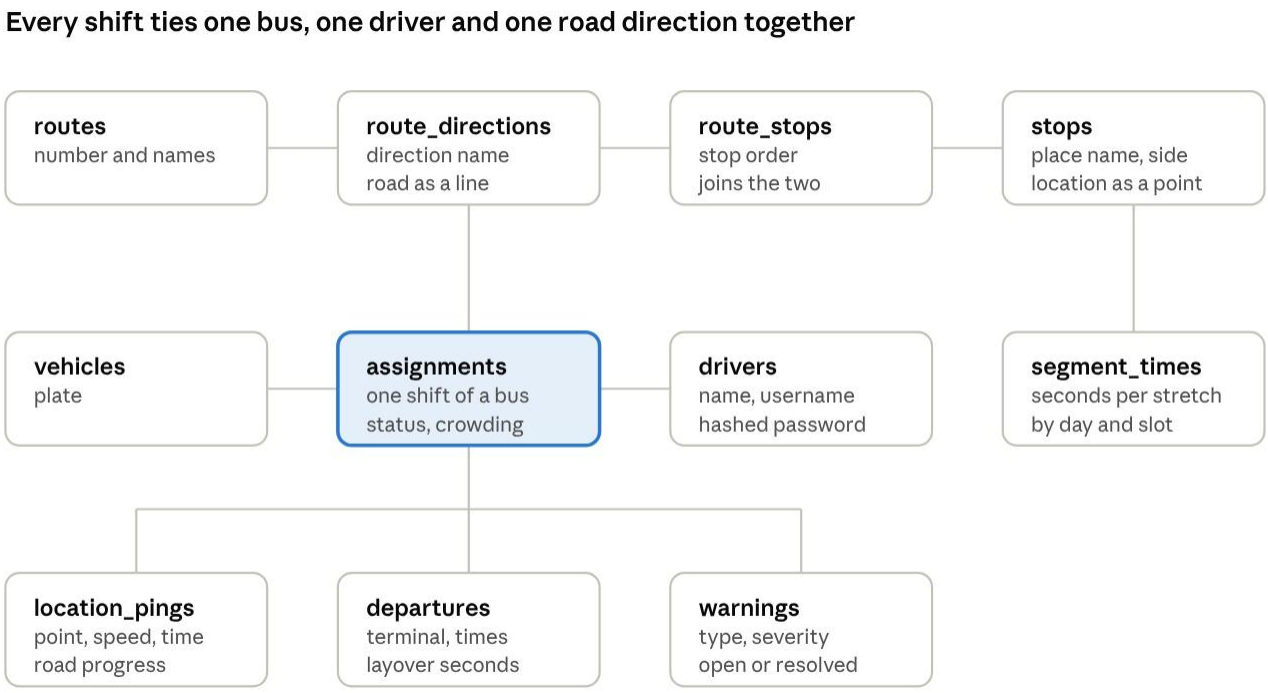
Three web apps talk to one backend, and only the backend talks to the database.



The driver app sends GPS points to the backend. The backend turns them into positions, arrival times and departures. The rider app only reads those results. The admin app manages drivers, vehicles and shifts. Two background jobs run inside the backend, and the database keeps the roads and stops as map data.

2. Database design

The database is PostgreSQL with the PostGIS extension, which stores roads as lines and stops as points.



main tables and how they connect

A route has two directions, and each direction has its own road and its ordered stops. An assignment is one shift: it joins a vehicle, a driver and a route direction. GPS points, terminal departures and warnings all belong to an assignment. Stretch times belong to pairs of stops, so every route that uses the same stretch shares them. Logins use the tables drivers , admins and two session tables that hold only token hashes.

3. API design

The backend is a REST API that speaks JSON. Riders need no login. Drivers and admins log in once and send a token with each request. Each feature has four files: routes, controller, service and repository.

Group	Endpoints	Used by	Login
Stop search	GET /stops?q=	Rider	None
Group	Endpoints	Used by	Login

Arrivals and departures	GET /stops/:id/arrivals	Rider	None
Trip planner	GET /trips?from=&to=	Rider	None
Live map	GET /routes/:id/live	Rider	None
Driver login	POST /auth/driver/login, GET /auth/me	Driver	Token
Shift control	GET /assignments/current, POST /assignments/select, start, return, departing, crowding, end	Driver	Token
Route list	GET /routes/directions	Driver	Token
GPS points	POST /pings (a batch of points)	Driver	Token
Admin login	POST /auth/admin/login, GET /auth/admin/me	Admin	Token
Admin management	/admin/drivers, /admin/vehicles, /admin/assignments (and /:id/end), /admin/route-directions	Admin	Token
Admin monitoring	GET /admin/live-all, GET /admin/assignmentchanges	Admin	Token

4. Key algorithms

Every estimate comes from one idea: place the bus on its road as a number from 0 to 1, then measure the distance left to the rider's stop.

Position on the road. Each route direction stores its road as a line. Each GPS point is matched to that line, which gives the bus's progress along it. The match only looks 500 m ahead of the last position and allows 50 m back for GPS noise. A point more than 100 m from the line counts as off the road.

Speed. The bus speed is the average over the last 45 seconds. At least 20 seconds of points are needed. Otherwise a fallback speed of 20 km/h is used and the range gets wider.

Arrival time (ETA). The time for each stretch between two stops comes from the table `segment_times` , ekend and by 30-minute slot. Every value starts as a

guess. A nightly job replaces a guess with the median of real crossings once a slot has at least 3 of them, and marks it measured. The answer is a range of plus or minus 20%, or 40% when the bus's own speed can't be used. An ETA of 60 seconds or less reads "Arriving".

Freshness and standing. A bus whose newest point is older than 90 seconds gets no estimate and turns grey on the map. A bus that stands still for at least 2 minutes shows "Standing". A red light is shorter than that and is ignored.

Terminals and departures. A bus within 100 m of a terminal is at the terminal. The terminal then shows when the next bus leaves. A bus is "departing soon" when it is full, when the crew taps Departing now, or after the usual 10-minute layover. It is "later than expected" after 25 minutes. When the bus starts the other road, the direction flips by itself after 3 points show progress on the opposite road (50 m or more), or when the crew taps Start return trip.

Shift lifecycle. A shift moves from assigned to on trip, to at terminal and back, then to ended. The server checks every 60 seconds. A shift with no GPS point for 30 minutes ends by itself.

Warnings. The system raises wrong way, off path, bad phone clock, no signal (180 seconds) and overdue at terminal. A bus with an open wrong-way or off-path warning is hidden from riders until it clears. All limits live in one file, placeholders.ts , so they can be cha ride.

5. Rider app UI design

The rider app has two tabs, Trips and Stops, over one map that is never thrown away. It is a mobile-first web app built with React, Tailwind CSS and Leaflet, and it can be added to a phone's home screen.

Trips tab. A teal header holds only the logo. Under it a rounded From and To card has place search, a swap button and Find bus. The real OpenStreetMap map fills the rest, with zoom buttons and a locate button that stay clear of the card and the tab bar. Riders pick places, not stops. The backend works out the stop and the direction from the order of the two places.

Results sheet. After Find bus, the results slide up as a sheet over the map. The map shows each route as a line in its own colour, with its buses.

Height	What the rider sees	How to get there
Peek	Map almost full screen and a one-line summary such as "2 routes · 4–7 min"	Drag the sheet down
Half	Map on top and the first route cards (the default)	After Find bus, or after tapping a card

Full	The list of routes fills the screen	Drag up, or tap the bar

Tapping a route card highlights its line and bus, fades the other routes, and returns the sheet to Half. Each card shows the route badge, the direction, the stops away, where to wait, the next bus time and tags for crowding and standing.

Stops tab. A stop search with recent stops, then a stop screen. The stop screen shows buses on the road, upcoming buses (grey, rough estimate) and, at a terminal, the departures: buses waiting, boarding now, time since the last bus left and the usual gap.

Staying in place. The search and the chosen route live in the page address, so a refresh or a switch between tabs keeps them. Data refreshes every 5 seconds on the map and every 10 seconds on a stop, and pauses while the tab is hidden.

Look and feel. Teal header (#3f7468), dark outlines on every card and button, white and cream surfaces, amber accents, and the Rubik font. Route 49 is red and route 61 is blue. Other routes get a fixed colour from a small palette. Grey means a bus with no recent signal.

6. Driver and admin app design

Driver app. A phone web app for the crew, built with React and plain CSS. The screens are Login, Setup (choose the road direction), Shift (start) and Running.

- **GPS sender:** it reads the phone position, keeps the points in the phone's storage and sends them in batches to POST /pings every 5 seconds, up to 200 points at a time. A point that cannot be sent stays queued and goes out when the connection returns. While the bus stands still, a heartbeat point goes out every 10 seconds so the bus does not look dead.
- **Crew buttons:** how full the bus is (seats available, standing only, full), Departing now, Start return trip and End shift. The screen shows the state of the shift.
- **Safety:** each point carries its own time, so a late batch still lands in the right place. The server ignores the timing of a batch if the phone clock is more than 3 minutes wrong.

Admin app. A web app for the operator, with a login and four pages:

Page	What it does
Live	A table of all running buses with their state and any warnings (for example no signal)
Vehicles	Add, edit and delete buses by plate

Drivers	Add, edit and delete drivers, and reset a password
Shifts	Assign a driver and a vehicle to a route direction, edit it, or end it

7. Security and data handling

- **Passwords:** driver and admin passwords are stored only as bcrypt hashes. A plain password is never saved.
- **Sessions:** a login gives a random token. Only a hash of the token is stored, and it expires after 14 hours, about one shift.
- **Access:** driver and admin endpoints check the token on every call, and admin endpoints accept only admin tokens. Rider endpoints are public and read-only.
- **Rider privacy:** riders send nothing about themselves. The rider app never uploads the rider's location. The locate button only moves the rider's own map.
- **Bad data:** the server drops points with poor accuracy and points from the future, and ignores batch timing when a phone clock is badly wrong.
- **Secrets:** the database address and the admin password live in environment variables on the host. The `.env` files are never committed.
- **Cross-origin access:** the API allows requests from the web apps through CORS.